AI/ ML Engineer Intern Assessment

ASSESSMENT INSTRUCTIONS

Position: AI/ ML Engineer Intern

- **Start Time: 27th March 2026, 2:00 PM (IST)**
- **Deadline: 30th March 2026, 2:00 PM (IST)**
- **DEADLINE - NO EXTENSIONS GRANTED**

Submission Requirements:

- **File Name:** AI/ ML Engineer Intern_[Your_Name] _March2026
- **Email to:** [career@purplemerit.com] with subject " AI/ML Engineer Intern Assessment Submission - [Your Name]"

❖ **LATE SUBMISSIONS ARE DISQUALIFIED.**

We've provided **two assessment options** (Assessment 1 and Assessment 2) below. Please choose **one** that best matches your strengths and submit that completed assessment. Completing **both** is optional; if you do, it will be considered additional signal in your evaluation.

Assessment Overview

Assessment – 1

Agentic RAG Challenge: Prerequisite & Course Planning Assistant (Catalog-Grounded)

This assessment evaluates your ability to build a **retrieval-augmented generation (RAG)** system (optionally agentic using **CrewAI** or **LangChain**) that answers student course-planning questions **strictly grounded** in program and course catalog documents. We are assessing correctness, grounding, citation quality, and your ability to handle ambiguity safely.

Scenario

You're building an assistant inside an LMS that helps students plan courses for the upcoming term. The assistant must interpret **program requirements** and **course prerequisites** from catalog documents and produce a proposed course plan.

This is intentionally tricky because catalogs contain:

- prerequisite chains (A → B → C)
- "either/or" requirements

- minimum grade requirements
- co-requisites
- exceptions ("instructor consent")
- program rules (credits, electives, residency requirements)
- inconsistent wording across pages

Objective

Build a **Prerequisite & Course Planning Assistant** that can:

1. Answer prerequisite questions with **verifiable citations**
2. Suggest a term plan given a student's completed course
3. Ask clarifying questions when student info is incomplete
4. Refuse to guess when policy is not in the documents

Data Requirements (You curate the catalog)

Use **publicly available** academic catalog documents (university/college catalog pages are fine).

Minimum dataset

- At least **20 course pages** (or course descriptions)
- At least **2 program/degree requirement pages** (major/minor/degree requirements)
- At least **1 academic policy page** (grading, repeats, credit limits, etc.)
- Total: **30,000+ words** OR **25+ distinct documents/pages**

Documentation

Include a "Sources" section with:

- URLs
- date accessed
- short note on what each source covers

Required Capabilities (Functional Requirements)

1) Grounded answers with citations (mandatory)

- Every prerequisite / rule / requirement claim must include citations (URL + section heading or chunk id).
- If you cannot cite it, do not state it as fact.

2) Prerequisite reasoning (mandatory)

Support questions like:

- "Can I take CS301 if I've taken CS101 and MATH120?"
- "What do I need before enrolling in Database Systems?"
- "Is co-requisite required concurrently or can it be completed earlier?"

You must show reasoning as:

- **Decision** (Eligible / Not eligible / Need more info)
- **Evidence** (citations)
- **Next step** (what to take/do)

3) Course plan generation (mandatory)

Given:

- completed courses (and optionally grades)
- target program/major
- target term (Fall/Spring)
- max courses/credits

Produce:

- a suggested course list for next term
- justification per course (why it fits requirements + prereq satisfied)
- "risks/assumptions" section (e.g., offering availability not in docs)

4) Clarifying questions (mandatory)

If required inputs are missing (major, grades, which catalog year, transfer credits, etc.), the assistant must ask **1–5 clarifying questions** before planning.

5) Safe abstention (mandatory)

If the documents do not contain the answer (e.g., course availability by semester), respond:

- "I don't have that information in the provided catalog/policies."
- Suggest what to check next (advisor, department page, schedule of classes)

Implementation Requirements

You may implement using **CrewAI** or **LangChain** (or equivalent), but your solution must include:

RAG pipeline

- Document ingestion (HTML/PDF/text)
- Cleaning + chunking strategy (explain chunk size/overlap)
- Embeddings model
- Vector store (FAISS/Chroma preferred)
- Retriever configuration (k, similarity metric)
- Prompt that enforces:
    - citations
    - "don't know" behavior
    - structured outputs

Agentic design (required if using CrewAI; optional if using LangChain)

If using **CrewAI**, include at least 3 agents such as:

- **Intake Agent**: collects missing student info; normalizes the student profile
- **Catalog Retriever Agent**: retrieves relevant program and course requirement text
- **Planner Agent**: proposes an eligible plan grounded in citations
- **Verifier/Auditor Agent** (recommended): checks for missing citations, unsupported claims, prereq mistakes

If using **LangChain**, you can implement equivalent stages as chains/tools (router + retrieval + verifier).

Output format (mandatory)

Every response must follow:

Answer / Plan:
Why (requirements/prereqs satisfied):
Citations:
Clarifying questions (if needed):
Assumptions / Not in catalog:

Evaluation (Required)

Create a test set of **at least 25 queries**, including:

1. **10 prerequisite checks** (eligible / not eligible)
2. **5 prerequisite chain questions** (multi-hop, needs 2+ courses of evidence)
3. **5 program requirement questions** (electives, credits, required categories)
4. **5 "not in docs" / trick questions** (availability, professor approval details, etc.)

Report:

- **Citation coverage rate** (% responses with citations)
- **Eligibility correctness** on the prereq checks (manual grading is OK—describe your rubric)
- **Abstention accuracy** on "not in docs"
- 3 example transcripts:
    1. correct eligibility decision with citations
    2. a course plan output with justification + citations
    3. correct abstention + guidance

Deliverables

1) Implementation (Required — GitHub)
Your submission must run end-to-end and include:

- building the index (ingestion → chunking → embeddings → vector store)
- running sample interactions
- running the evaluation set and outputting results

Submit: GitHub repository link (with a clear README + setup/run steps) and a short write-up (PDF or Google Doc link).

2) Short Write-Up (Max 1 page)
Include:

- chosen catalog/institution + sources (URLs + date accessed)
- architecture overview (diagram optional)
- chunking/retrieval choices and tradeoffs
- prompts and/or agent roles (high level)
- evaluation summary, key failure modes, and next improvements

3) Optional Demo (Recommended)

- Gradio/Streamlit demo **or** a **1–2 minute** screen recording showcasing the assistant end-to-end (including citations and one edge case).

Assessment – 2

E-commerce Support Resolution Agent

This assessment evaluates your ability to build a **multi-agent RAG system** (CrewAI recommended; LangChain/LangGraph acceptable) that resolves customer support tickets using policy documents, with strong controls against hallucination, missing citations, and unsafe outputs.

Scenario

You are building an internal AI assistant for an e-commerce customer support team. The assistant must read policy documentation and generate **customer-ready resolutions** for incoming tickets.

Tickets often involve:

- policy exceptions (final sale, hygiene items, perishables, opened items)
- regional differences (country/state rules)
- marketplace complexity (seller vs first-party)
- ambiguity (missing order status, item category, delivery date)
- user attempts to pressure support for outcomes not covered by policy

Your assistant must be helpful, accurate, and grounded—**without inventing policy**.

Objective

Build a multi-agent system that, given a support ticket + limited order context, produces a structured resolution that is:

- **policy-grounded**
- **citation-backed**
- **safe and compliant**
- robust to missing/ambiguous information

Required Agent Design (Minimum 4 Agents)

Implement at least the following agents (names can vary, responsibilities must match):

1. **Triage Agent**
    - Classifies issue type (refund / shipping / payment / promo / fraud / other)
    - Identifies missing fields and asks clarifying questions (if required)
2. **Policy Retriever Agent**
    - Queries a vector database
    - Returns only the most relevant excerpts
    - Includes citations (doc title/URL + section or chunk id)
3. **Resolution Writer Agent**
    - Drafts the final customer support response
    - Must only use retrieved evidence (no unsupported claims)
4. **Compliance / Safety Agent**
    - Checks for:
        - unsupported statements
        - missing/weak citations
        - policy violations
        - sensitive data leakage
    - Can force a rewrite or trigger an abstain/escalation path

You may add additional agents (e.g., Order Context Interpreter, Fraud Redactor, Escalation Agent) if useful.

Data Requirements (Policy Corpus)

Use a policy knowledge base built from **public e-commerce policy documents** (or a synthetic internal policy pack you author yourself).

Minimum requirements:

- **12+ documents** OR **25,000+ words** total
- Must cover:
    1. returns & refunds (including exceptions)
    2. cancellations
    3. shipping & delivery / lost package
    4. promotions / coupon terms
    5. disputes (damaged/incorrect item, missing items)

Include a **Sources** section (URLs + date accessed). Use only permissible data.

Input Format (You define and document it)

Your system must accept, at minimum:

A) Ticket text (free form)

Example:

"My order arrived late and the cookies are melted. I want a full refund and to keep the item."

B) Order context (structured JSON)

Include at least:

- order_date, delivery_date (if delivered)
- item_category (e.g., perishable / apparel / electronics)
- fulfillment_type (first-party vs marketplace seller)
- shipping_region
- order_status (placed/shipped/delivered/returned)

- payment_method (optional)

You may create synthetic order contexts for your evaluation set.

Required Output (Structured)

For every ticket, the system must output:

1. **Classification:** issue type + confidence
2. **Clarifying Questions (if needed):** max 3
3. **Decision:** approve/deny/partial/needs escalation
4. **Rationale:** policy-based explanation
5. **Citations:** bullet list with doc + section/chunk id
6. **Customer Response Draft:** customer-ready message
7. **Next Steps / Internal Notes:** what the agent recommends support does next

Hard rule: Any policy claim must be backed by citations.

Tricky Requirements (Must Handle)

Your test set must include cases with:

- **Conflicting policies** (region vs seller vs category differences)
- **Exception-heavy categories** (final sale, hygiene, perishables, opened items)
- **Ambiguous tickets** that require clarifying questions
- **"Not in policy" requests** (must abstain or escalate; no guessing)

Implementation Requirements

You may use CrewAI or LangChain/LangGraph. Your submission must include:

- Document ingestion pipeline (HTML/PDF/text)
- Chunking strategy (explain chunk size/overlap and why)
- Embeddings model selection
- Vector store (FAISS/Chroma preferred)
- Retriever settings (top-k, filters/metadata if used)

- Prompts for each agent
- A “no hallucination” control mechanism, such as:
    - evidence-only generation prompt + refusal rules
    - minimum evidence threshold before answering
    - verifier agent that blocks unsupported claims
    - citation checker that fails the run if citations are missing

Evaluation (Required)

Create a test set of **at least 20 tickets**:

- 8 standard cases
- 6 exception-heavy cases
- 3 conflict cases
- 3 not-in-policy cases

Report:

- **Citation coverage rate** (% outputs with citations)
- **Unsupported claim rate** (manual review is fine—define rubric)
- **Correct escalation rate** (for conflict/not-in-policy)
- Provide **3 full example runs**:
    1. exception handled correctly
    2. conflict handled with escalation
    3. correct abstention / “need more info” path

Deliverables

1. **GitHub Repository (Required)**
   Include:
    - Clear README with setup/run instructions
    - Code for ingestion, indexing, agent orchestration, and evaluation
    - Example inputs/outputs

- Requirements file (requirements.txt / pyproject)

2. **Short Write-Up (Max 1 page)**
   Include:
   - Architecture overview (diagram optional)
   - Agent responsibilities + prompts (high level)
   - Data sources
   - Evaluation summary + key failure modes
   - What you would improve next
3. **Product Demo Video (Optional but Recommended)**
   - A **1–2 minute** screen recording showcasing:
     - one end-to-end ticket resolution
     - citations shown in the output
     - how conflict/exception cases are handled

Good luck—we're looking forward to reviewing your submission. There's no single "right" answer; we're evaluating clear engineering thinking, sound technical tradeoffs, reliability and grounding, thoughtful evaluation, and your ability to build a practical solution end-to-end.